

Power maximization as an alternative assistant objective in MBAG

A controlled objective-swap inside AssistanceZero

EMPO fork of `minecraft-building-assistance-game`

Built on real `MbagEnv` + the trained checkpoint

Where this sits in your framework

- **Assistance games**: a robot helps a human whose goal is hidden. (You know this — this is not a tutorial.)
- **AssistanceZero** solves it by **goal inference**: maintain a belief over goals, maximize completion of the inferred goal.
- This deck: take the **power-maximization** idea (Heitzig & Potham, arXiv:2508.00159) and plug it into *your* framework as an **alternative assistant objective** — the robot gets *no extrinsic reward* and *never infers* the goal.

The claim of this talk

We integrated the power objective into MBAG *faithfully* — same env, same planner, same human prior — and the behavioral contrast is sharp, interesting, and (we think) worth your attention.

Two objectives, one environment

Both assistants are **model-based planners** over the known simulator (the AlphaZero idea).

They differ in *one* line:

Goal-inference (AssistanceZero)

Pick a guessed goal g^* ; maximize the human's attainment of *that* goal:

$$\max_a V_h^e(s_a, g^*).$$

Power (Heitzig & Potham)

Maximize aggregate human power over *all* possible goals, goal-agnostically:

$$\max_a U_r(s_a) \quad (\text{no } g^*).$$

Same MbagEnv, same goal-directed human prior π_h , same goal set G . **Only the objective changes** — so every behavioral difference is attributable to the objective.

The power objective: $V_h^e \rightarrow X_h \rightarrow U_r$

- **Effective goal-attainment** — how much of goal g the human reaches under π_h from the state the assistant leaves:

$$V_h^e(s, g) \in [0, 1].$$

- **Individual power** — a soft count of attainable goals:

$$X_h = \sum_{g \in G} V_h^e(s, g)^\zeta.$$

- **Aggregate reward** — inequality- & risk-averse over humans (single human here):

$$U_r = -\left(\sum_h X_h^{-\xi}\right)^\eta.$$

Paper's gridworld params (Table 5): $\zeta = 2$, $\xi = 1$, $\eta = 1.1$. The robot maximizes the human's ability to reach *any* goal — it is goal_agnostic: it never asks "which goal?".

How it plugs into MBAG (respecting your design)

- **Your environment.** The real MbagEnv (`set_state`, `step`, `goal_percentage`) — not a toy.
- π_h = **your human model.** A goal-directed builder fills the π_h slot; we verified your *trained* human and assistant policies load from `data/assistancezero_assistant/checkpoint_002000` (`load_checkpoint.py`) — same slot, learned, at scale.
- **Your planner.** AlphaZero-style model-based planning: score candidate help by *rolling* π_h *forward* in the real env.
- **The fork's substrate.** A `goal_agnostic` flag (zeroes/freezes the goal head) enforces the no-inference premise; **bedrock** (breakable only by the robot) and **clutter** are the affordances that reveal the difference.

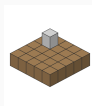
Experimental design

- A **controlled objective-swap**: three arms — no assistant, **goal-inference**, **power** — on the *same* state, goal set, and human.
- **No training**. Each assistant is a planner: it enumerates a small set of candidate “help” plans, applies each to the world, and scores it by its objective via real π_h rollouts.
- Two vignettes, each engineered so the objectives *must* diverge:
 - V1 **open the options** — bedrock obstacles only the robot can clear.
 - V2 **don't foreclose** — committing to a guess can foreclose the other goals.

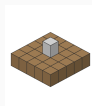
Code: `empo_real/vignettes.py`. The numbers and figures on the next slides are produced by running it directly.

V1 — “open the options” (bedrock asymmetry)

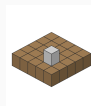
Every goal = one stone block, blocked by bedrock only the robot can clear. True goal = **G2** (hidden).



G0



G1 (guess)



G2 (true)

goal-inference: clears only its guess



$X_h = 1$, human **fails** true goal ($V_h^e = 0$)

power: clears all



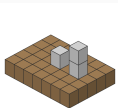
$X_h = 3$, human **succeeds** ($V_h^e = 1$)

Takeaway. The goal-inference robot serves its *guess* and leaves the true goal walled off. The power robot opens *every* option — so whatever the human wants, it is reachable.

V2 — “don’t foreclose” (disempowerment when wrong)

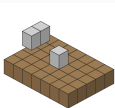
Goals A and B share a foundation. The robot can build the foundation (helps both) or commit toward A. True goal = **G1 = B** (hidden).

no assistant



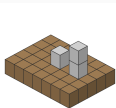
$X_h = 1.25$
ok ($V_h^e = 1$)

goal-inference



commits to A
only 43% ($V_h^e = 0.43$)

power



builds foundation
succeeds ($V_h^e = 1$)

The sharp one. Guessing wrong, the goal-inference robot pours effort into A; the human must *undo* it and runs out of time — **worse than no help at all** ($1.18 < 1.25$). This is exactly the *disempowerment-when-wrong* the power objective avoids by keeping options open.

Results summary

Vignette	Arm	plan	X_h	true-goal V_h^e
V1 open the options	no assistant	noop	0.00	0.00 fails
	goal-inference	clear G1	1.00	0.00 fails
	power	clear all	3.00	1.00 succeeds
V2 don't foreclose	no assistant	noop	1.25	1.00 ok
	goal-inference	build toward A	1.18	0.43 fails
	power	build foundation	1.36	1.00 succeeds

$\zeta = 2$, $\xi = 1$, $\eta = 1.1$. Figures and numbers regenerated by `empo_real/vignettes.py` (and `presentation/make_figures.py`). In V2 the goal-inference arm leaves the human *below* the no-help baseline.

Why it matters

- **Empowerment instead of inference.** The robot need not be *right* about the goal — it keeps the human able to reach any of them.
- **Robust to wrong beliefs.** Goal inference fails gracelessly when the belief is wrong (V2): effort spent on the wrong goal actively *disempowers*. Power degrades gracefully.
- **Avoid irreversible disempowerment.** The power objective is explicitly inequality- and risk-averse (U_r) — it resists foreclosing options, which is the corrigibility / safety motivation of the paper.
- **Complementary to AssistanceZero**, not a replacement: same env, same human, swappable objective — an interesting alternative for your framework to study.

Honest scope & next steps

What this is

A faithful *planning-based* integration of the power objective into MBAG; a controlled objective comparison with your real components. No training.

What this is not (yet)

A trained head-to-head at full $11 \times 10 \times 10$ scale. That is the labelled next phase (EMPO_PLAN Phase 2–3):

- **Phase 2:** amortize the planner's V_h^e into a learned network.
- **Phase 3:** train the power-maximizing assistant via your AlphaZero loop; head-to-head vs. the trained AssistanceZero assistant.

Conclusion

The power objective slots cleanly into AssistanceZero —
same env, same human, same planner.

When the robot's belief is wrong, **goal inference** can
disempower; **power** keeps the human's options open.

An interesting alternative assistant objective for your framework.